

19 – Inheritance

Herencia

Muchos tipos de datos tienen propiedades en común con otros tipos. Por ejemplo, los tipos `list` y `str` cada uno tienen funciones `len` que significan lo mismo. La **herencia** proporciona un *mecanismo conveniente para construir grupos de abstracciones relacionadas*. Permite a los programadores crear una jerarquía de tipos en la cual cada tipo hereda atributos de los tipos por encima de él en la jerarquía.

La **programación orientada a objetos** (POO) permite *representar relaciones del mundo real*. Al crear jerarquías de clases, podemos agrupar distintos objetos bajo un mismo tipo, compartiendo comportamientos y atributos. Esto nos permite escribir código más organizado, reutilizable y fácil de mantener.

La **clase** `object` *está en la parte superior de la jerarquía*. Esto tiene sentido, ya que en Python todo lo que existe en tiempo de ejecución es un objeto. Debido a que `Person` hereda todas las propiedades de los objetos, los programas pueden vincular una variable a una `Person`, agregar una `Person` a una lista, etc.

La clase `MIT_person` en el código a continuación hereda atributos de su clase padre, `Person`, incluyendo todos los atributos que `Person` heredó de su clase padre, `object`. En la jerga de la programación orientada a objetos, `MIT_person` es una **subclase** de `Person`, y por lo tanto **hereda** los atributos de su **superclase**. Además de lo que hereda, la subclase puede:

- **Agregar nuevos atributos**. Por ejemplo, la subclase `MIT_person` ha agregado la variable de clase `_next_id_num`, la variable de instancia `_id_num`, y el método `get_id_num`.
- **Sobrescribir**, es decir, reemplazar, **atributos de la superclase**. Por ejemplo, `MIT_person` ha sobrescrito `__init__` y `__lt__`. Cuando un método ha sido sobrescrito, la versión del método que se ejecuta se basa en el objeto usado para invocar el método. Si el tipo del objeto es la subclase, la versión definida en la subclase será usada. Si el tipo del objeto es la superclase, la versión en la superclase será usada.

El método `MIT_person.__init__` primero usa `super().__init__(name)` para invocar la función `__init__` de su superclase (`Person`). Esto

inicializa la variable de instancia heredada `self._name`. Luego inicializa `self._id_num`, que es una variable de instancia que las instancias de `MIT_person` tienen pero las instancias de `Person` no tienen.

La variable de instancia `self._id_num` se inicializa usando una **variable de clase**, `_next_id_num`, que pertenece a la clase `MIT_person`, en lugar de a instancias de la clase. Cuando se crea una instancia de `MIT_person`, no se crea una nueva instancia de `next_id_num`. Esto permite a `__init__` asegurar que cada instancia de `MIT_person` tenga un único `_id_num`.

Es buena práctica usar `super()` para llamar al constructor de la clase padre, ya que garantiza que los atributos heredados se inicialicen correctamente, incluso si en el futuro se cambia la jerarquía de clases.

Clase MIT_person

```
1  class MIT_person(Person):
2      _next_id_num = 0 #número de identificación
3
4      def __init__(self, name):
5          super().__init__(name)
6          self._id_num = MIT_person._next_id_num
7          MIT_person._next_id_num += 1
8
9      def get_id_num(self):
10         return self._id_num
11
12     def __lt__(self, other):
13         return self._id_num < other._id_num
```

Considera el código

```
1  p1 = MIT_person('Barbara Beaver')
2  print(str(p1) + '\n's id number is ' + str(p1.get_id_num()))
```

Si una clase no define un método `__str__()`, Python busca si alguna de sus superclases lo tiene. Este comportamiento se aplica a cualquier método, se usa el primero que se encuentre subiendo por la jerarquía de herencia.

La primera línea crea una nueva `MIT_person`. La segunda línea es más complicada. Cuando intenta evaluar la expresión `str(p1)`, el sistema de tiempo de ejecución primero verifica si hay un método `__str__` asociado con la clase `MIT_person`. Como no lo hay, luego verifica si hay un método `__str__` asociado con la superclase inmediata, `Person`,

de `MIT_person`. Lo hay, así que usa ese. Cuando el sistema de tiempo de ejecución intenta evaluar la expresión `p1.get_id_num()`, primero verifica si hay un método `get_id_num` asociado con la clase `MIT_person`. Lo hay, así que invoca ese método e imprime

```
1 Barbara Beaver's id number is 0
```

(Recuerda que en una cadena, el carácter `'\'` es un carácter de escape usado para indicar que el siguiente carácter debe ser tratado de manera especial. En la cadena

```
1 '\s id number is '
```

el `'\'` indica que el apóstrofe es parte de la cadena, no un delimitador que termina la cadena.)

Ahora considera el código

```
1 p1 = MIT_person('Mark Guttag')
2 p2 = MIT_person('Billy Bob Beaver')
3 p3 = MIT_person('Billy Bob Beaver')
4 p4 = Person('Billy Bob Beaver')
```

Hemos creado cuatro personas virtuales, tres de las cuales se llaman Billy Bob Beaver. Dos de los Billy Bobs son de tipo `MIT_person`, y uno es meramente una `Person`. Si ejecutamos las líneas de código

```
1 print('p1 < p2 =', p1 < p2)
2 print('p3 < p2 =', p3 < p2)
3 print('p4 < p1 =', p4 < p1)
```

el intérprete imprimirá

```
1 p1 < p2 = True
2 p3 < p2 = False
3 p4 < p1 = True
```

Como `p1`, `p2`, y `p3` son todos de tipo `MIT_person`, el intérprete usará el método `__lt__` definido en la clase `MIT_person` cuando evalúe las primeras dos comparaciones, así que el ordenamiento se basará en números de identificación. En la tercera comparación, el operador `<` se aplica a operandos de diferentes tipos. Como el primer argumento de la expresión usó para determinar cuál método `__lt__` invocar, `p4 < p1` es una forma abreviada para `p4.__lt__(p1)`. Por lo tanto, el intérprete usa el método `__lt__` asociado con el tipo de `p4`, `Person`, y las "personas" serán ordenadas por nombre.

Qué pasa si tratamos:

```
1 print('p1 < p4 =', p1 < p4)
```

El sistema de tiempo de ejecución invocará el operador `__lt__` asociado con el tipo de `p1`, es decir, el definido en la clase `MIT_person`. Esto llevará a la excepción

```
1 AttributeError: 'Person' object has no attribute '_id_num'
```

porque el objeto al cual `p4` está vinculado no tiene un atributo `_id_num`.

El uso de `isinstance(obj, Clase)` permite verificar si un objeto pertenece a una clase o a alguna de sus subclases. Es una forma más flexible y robusta que `type(obj) == Clase`, ya que respeta la jerarquía de herencia.

Ejercicio manual

Implementa una subclase de `Person` que satisfaga la especificación

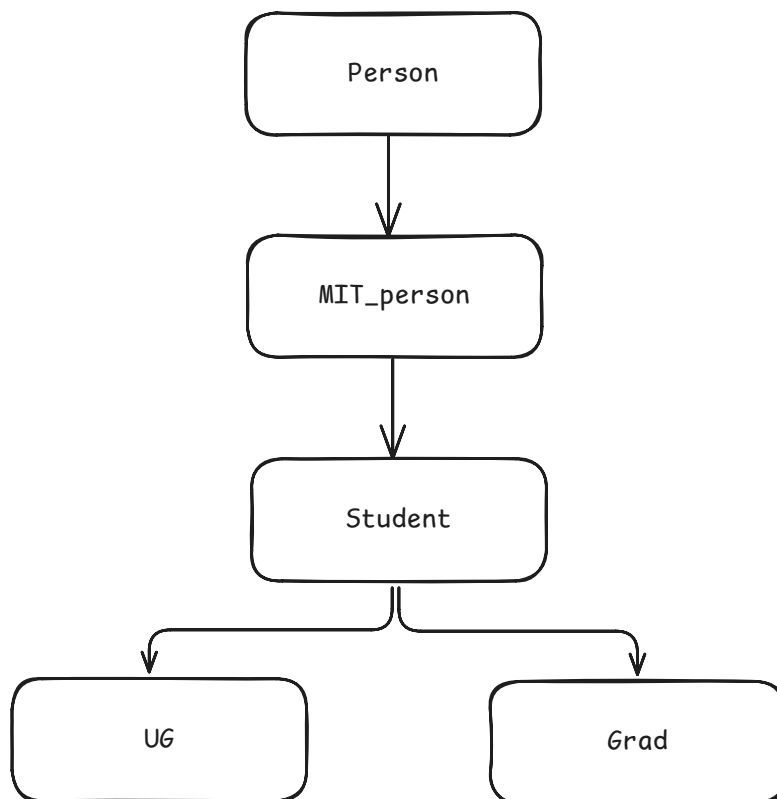
agrega otro par de niveles de herencia a la jerarquía de clases.

```
1 class Politician(Person):
2     """Un político es una persona que puede pertenecer a un
3     partido político"""
4
5     def __init__(self, name, party = None):
6         """name y party son cadenas"""
7         super().__init__(name)
8         self.party = party
9
10    def get_party(self):
11        """Devuelve el partido al cual self pertenece"""
12        return self.party
13
14    def might_agree(self, other):
15        """Devuelve True si self y other pertenecen al mismo partido
16        o al menos uno de ellos no pertenece a un partido"""
17        if (
18            self.party == other.get_party()
19            or self.party is None
20            or other.get_party() is None
21        ):
22            return True
23        return False
```

Múltiples Niveles de Herencia

Dos tipos de estudiantes

```
1 class Student(MIT_person):
2     pass
3
4 class UG(Student):
5     def __init__(self, name, class_year):
6         super().__init__(name)
7         self._year = class_year
8     def get_class(self):
9         return self._year
10
11 class Grad(Student):
12     pass
```



Agregar `UG` parece lógico, porque queremos asociar un año de graduación (o tal vez anticipada graduación) con cada estudiante universitario. Pero ¿qué está pasando con las clases `Student` y `Grad`? Al usar la palabra reservada de Python `pass` como cuerpo, indicamos que la clase no tiene atributos distintos a los heredados de su superclase. ¿Por qué alguien querría crear una clase sin nuevos atributos?

Incluso si una subclase no agrega nuevos atributos, como `Grad`, puede ser útil definirla para distinguir tipos de objetos y usarlos en validaciones o decisiones lógicas en el código.

Al introducir la clase `Grad`, ganamos la habilidad de crear dos tipos de estudiantes y usar sus tipos para distinguir un tipo de objeto de otro. Por ejemplo, el código

```
1 p5 = Grad('Buzz Aldrin')
2 p6 = UG('Billy Beaver', 1984)
3 print(p5, 'is a graduate student is', type(p5) == Grad)
4 print(p5, 'is an undergraduate student is', type(p5) == UG)
```

imprimirá

```
1 Buzz Aldrin is a graduate student is True
2 Buzz Aldrin is an undergraduate student is False
```

La utilidad del tipo intermedio `Student` es más sutil. Considera volver a la `class MIT_person` y agregar el método

```
1 def is_student(self):
2     return isinstance(self, Student)
```

La función `isinstance` está incorporada en Python. El primer argumento de `isinstance` puede ser cualquier objeto, pero el segundo argumento debe ser un objeto de tipo `type` o una tupla de objetos de tipo `type`. La función devuelve `True` si y solo si el primer argumento es una instancia del segundo argumento (o, si el segundo argumento es una tupla, una instancia de uno de los tipos en la tupla). Por ejemplo, el valor de `isinstance([1,2], list)` es `True`.

Volviendo a nuestro ejemplo, el código

```
1 print(p5, 'is a student is', p5.is_student())
2 print(p6, 'is a student is', p6.is_student())
3 print(p3, 'is a student is', p3.is_student())
```

imprime

```
1 Buzz Aldrin is a student is True
2 Billy Beaver is a student is True
3 Billy Bob Beaver is a student is False
```

Nota que el significado de `isinstance(p6, Student)` es bastante diferente del significado de `type(p6) == Student`. El objeto al cual `p6` está vinculado es de tipo `UG`, no `Student`. El objeto al cual `p6` está vinculado es una instancia de `Student`, pero dado que `UG` es una subclase de `Student`, el objeto al cual `p6` está vinculado es una

instancia de la clase `Student` (así como una instancia de `MIT_person` y `Person`).

Dado que solo hay dos tipos de estudiantes, podríamos haber implementado `is_student` como,

```
1 def is_student(self):
2     return type(self) == Grad or type(self) == UG
```

Sin embargo, si más tarde se agregara un nuevo tipo de estudiante, sería necesario volver atrás y editar el código que implementa `is_student`. Al introducir la clase intermedia `Student` y usar `isinstance`, evitamos este problema. Por ejemplo, si fuéramos a agregar

```
1 class Transfer_student(Student):
2     def __init__(self, name, from_school):
3         MIT_person.__init__(self, name)
4         self._from_school = from_school
5
6     def get_old_school(self):
7         return self._from_school
```

no se necesita hacer cambios a `is_student`.

No es inusual durante la creación y posterior mantenimiento de un programa volver atrás y agregar nuevas clases o nuevos atributos a clases antiguas. Los buenos programadores diseñan sus programas de manera que minimicen la cantidad de código que podría necesitar ser cambiado cuando eso se hace.

Ejercicio manual

¿Cuál es el valor de la siguiente expresión?

```
1 isinstance('ab', str) == isinstance(str, str)
```

El valor de la expresión es `False`.

Esto se debe a que `'ab'` sí es una instancia de `str`, por lo que `isinstance('ab', str)` devuelve `True`.

En cambio, `str` es una clase (específicamente, una instancia de la clase `type`), no una cadena de texto, por lo tanto `isinstance(str, str)` devuelve `False`.

Como `True == False` es `False`, el resultado total de la expresión es `False`.

El Principio de Sustitución

Cuando se usa la subclasificación para definir una jerarquía de tipos, *las subclases deben considerarse como extensiones del comportamiento de sus superclases*. Hacemos esto agregando nuevos atributos, o anulando atributos heredados de una superclase. Por ejemplo, `ReadingStudent` extiende `Student` introduciendo una antigua escuela.

A veces, la subclase anula métodos de la superclase, pero esto debe hacerse con cuidado. En particular, *los comportamientos importantes del supertipo deben ser respaldados por cada uno de sus subtipos*. Si el código del cliente funciona correctamente usando una instancia del supertipo, también debería funcionar correctamente cuando se sustituye una instancia del subtipo por la instancia del supertipo (de ahí la frase principio de sustitución). Por ejemplo, debería ser posible escribir código cliente usando la especificación de `Student` y hacer que funcione correctamente en un `TransferStudent`.

Por el contrario, no hay razón para esperar que el código escrito para funcionar con `TransferStudent` deba funcionar para tipos arbitrarios de `Student`.

Encapsulamiento y ocultamiento de información

Una buena práctica en programación orientada a objetos es no acceder directamente a los atributos internos de una clase desde fuera de ella. En lugar de eso, se recomienda usar métodos específicos para obtener o modificar esos valores –los famosos getters y setters– como `get_age()` o `set_name()`.

Esto se conoce como **encapsulamiento**, y permite que la implementación interna de una clase pueda cambiar sin que eso afecte al resto del programa. Por ejemplo, si una clase deja de usar `self.age` y pasa a usar `self.years`, todo el código externo que usaba `a.age` se rompe, pero si usaba `a.get_age()`, seguirá funcionando sin problemas.

Este principio también se relaciona con el **ocultamiento de información**, que sugiere proteger los detalles internos de la clase para que solo se acceda a ellos mediante interfaces definidas. Así se reduce el riesgo de errores, se mejora el mantenimiento del código, y se evita que otros programadores (o nosotros mismos en el futuro) dependan de aspectos que podrían cambiar.

Aunque Python técnicamente permite acceder y modificar atributos desde fuera (por ejemplo, `a.age = 42`), hacerlo va en contra del estilo limpio y robusto que promueve la P00.

- Los atributos (datos y métodos) deben accederse a través de métodos (getters/setters), no directamente.
- Las subclasses pueden extender o sobrescribir el comportamiento de las superclases.
- Las variables de clase (como contadores de ID) son compartidas entre todas las instancias.
- El principio de sustitución establece que una subclase debe poder usarse en lugar de su superclase sin alterar el comportamiento del programa.

Ejercicio manual de la lectura

En este problema, implementarás dos clases según la especificación a continuación: una clase `Container` y una clase `Stack` (una subclase de `Container`).

Nuestra clase `Container` inicializará una lista vacía. Los dos métodos que tendremos son para calcular el tamaño de la lista y para agregar un elemento. El segundo método será heredado por la subclase. Ahora queremos crear una subclase para que podamos agregar más funcionalidad –la capacidad de remover elementos de la lista. Una `Stack` agregará elementos a la lista de la misma manera, pero se comportará de manera diferente al remover un elemento.

Una pila (stack) es una estructura de datos de último en entrar, primero en salir. Piensa en una pila de panqueques. Mientras haces panqueques, creas una pila de ellos con los panqueques más viejos en la parte inferior y los más nuevos en la parte superior. Al comenzar a comer los panqueques, tomas uno de la parte superior para remover el panqueque más nuevo de la pila. Al implementar tu clase `Stack`, tendrás que pensar sobre cuál extremo de tu lista contiene el elemento que ha estado en la lista durante el menor tiempo. Este es el elemento que querrás remover y retornar.

```
1 class Container(object):
2     """
3     A container object is a list and can store elements of any type
4     """
5     def __init__(self):
```

```

6         """
7         Initializes an empty list
8         """
9         self.myList = []
10
11     def size(self):
12         """
13         Returns the length of the container list
14         """
15         return len(self.myList)
16
17     def add(self, elem):
18         """
19         Adds the elem to one end of the container list, keeping the end
20         you add to consistent. Does not return anything
21         """
22         self.myList.append(elem)
23
24     class Stack(Container):
25         """
26         A subclass of Container. Has an additional method to remove elements.
27         """
28         def remove(self):
29             """
30             The newest element in the container list is removed
31             Returns the element removed or None if the queue contains no
elements
32             """
33             if len(self.myList) == 0:
34                 return None
35             else:
36                 return self.myList.pop()

```